OCP GLOBAL SUMMIT 2026 • ERIDU AI

高基数交换机与精确时间

> Dan Biederman（Eridu AI 首席技术专家）
> 用单跳拓扑与到达时间调度压缩微秒级延迟

时长 13 分 20 秒

来源 youtube.com/watch?v=P0lumqNAl5U

高基数交换机与精确时间

> **整理说明**　本文是 OCP Global Summit 2026 一场 13 分 20 秒演讲的图文实录。体例为：每页幻灯片 → 该页停留期间的**完整中文译文**（逐句对应，不删减、不概括）→ 英文原句 → 图下一句小结。分节按幻灯片切换顺序，标题不含时间戳。

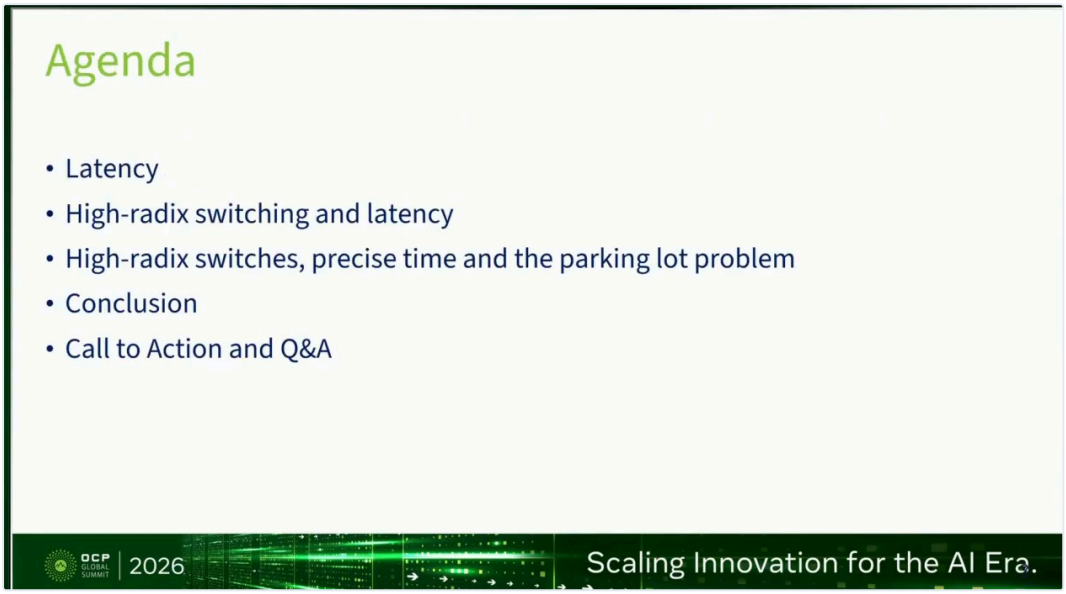
讲者　**Dan Biederman** — Principal Technologist, Eridu AI（首席技术专家）

这场演讲的位置　这是整场 TAP（时间设备项目）议程里唯一一场把"网络架构"和"精确时间"绑在一起讲的演讲。论点很集中：AI 负载的时间尺度是微秒，而今天的交换机把微秒耗在了**跳数**和**调度**上；高基数交换机消灭跳数，精确时间消灭调度不公，两者合起来才能把微秒拿回来。讲者开场即声明"这些主要是我的观点，不代表其他人"。

关于自动字幕的更正

字幕原文	更正为	说明
high rate / high radic / high radex / highic	high-radix	高基数（端口数极多）交换机
microscond / microsconds / nancond	microsecond / nanosecond	微秒 / 纳秒
Ir / IDO / Irdo / Erdo	Eridu	讲者所属公司 Eridu AI
one pabit / pedabit	one petabit	1 Pbps
102 t or terabits	102T（102 Tbps）	幻灯片标注
roundroin	round-robin	轮询调度
deficit weighted roundroin	deficit weighted round-robin（DWRR）	亏空加权轮询
nomalized	normalized	归一化
sinky	SyncE	Synchronous Ethernet，同步以太网
Iraq based time	rack-based time	机架级时间
nicks	NICs	网卡
multiplaner	multi-plane	多平面

01　议程：从延迟到停车场



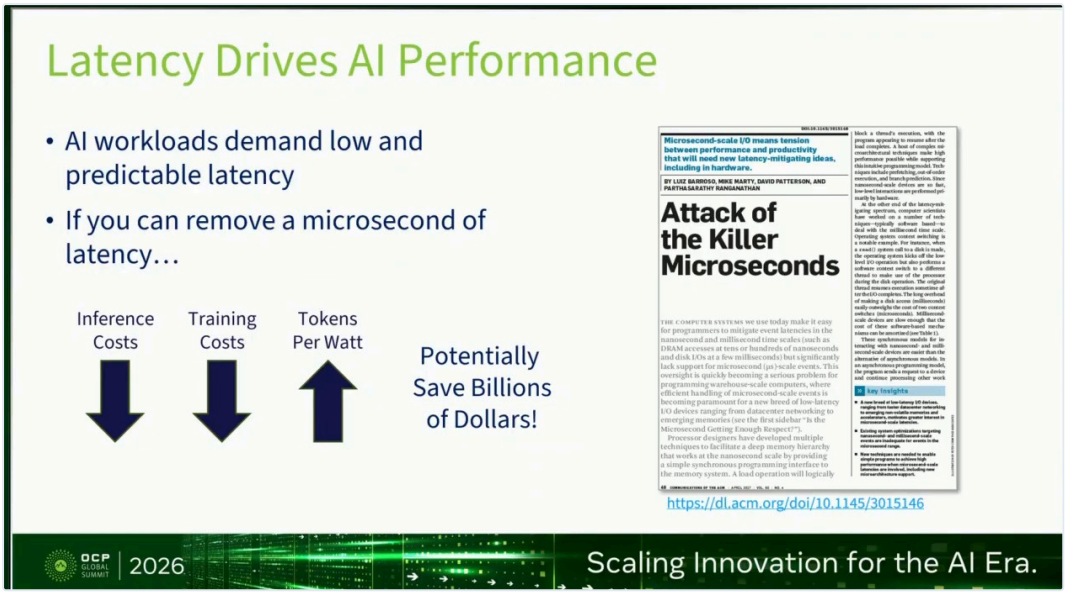
Agenda：Latency、High-radix switching and latency、High-radix switches, precise time and the parking lot problem、Conclusion、Call to Action and Q&A

好，今天我们要讲的是高基数交换机——它是什么——以及精确时间。先声明一下：这里面很多是我的观点，可能不代表其他人的观点。我们会讲延迟、高基数交换机与延迟，会讲高基数交换机、精确时间以及"停车场问题"，然后是结论、行动号召和问答。

> All right. So, today we're going to talk about high rate switches, what they are, and precise time. And so, just a disclaimer that a lot of these are my views and may not represent other people's views. Um, we'll talk about latency, high switches, and latency. We'll talk about high radics latency uh or high radic switches, precise time in the parking lot problem. and then we'll talk about Q&A or conclusion, call to action, and Q&A.

小结　议程页把"停车场"和"精确时间"并列，说明这不是一个比喻修辞，而是他真正要论证的调度模型——后面会用仿真数据来支撑。

02　延迟决定 AI 性能



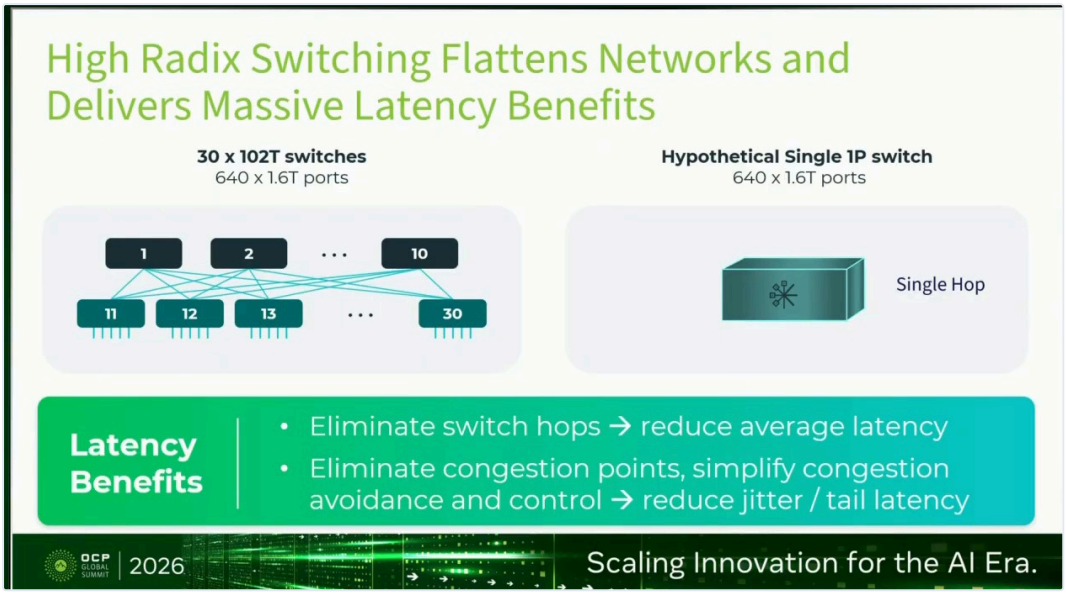
Latency Drives AI Performance：AI 负载需要低且可预测的延迟；去掉一微秒延迟可降低推理成本与训练成本、提升每瓦 token 数，可能省下数十亿美元；右侧为 CACM 论文《Attack of the Killer Microseconds》

对认识我一段时间的人来说，你们知道时间对我非常重要。对我的生活和职业影响最大的一篇论文，就是"致命微秒"（killer microseconds）这个概念。那篇论文讲的是：我们很擅长处理毫秒级问题，也很擅长处理纳秒级问题，但我们不擅长处理微秒级问题。而今天 AI 负载恰恰工作在微秒尺度上。所以如果我们能把这些微秒从延迟里拿掉，我们就能降低推理成本和训练成本、提升每瓦 token 数，并有可能为数据中心提供商或 AI 提供商省下数十亿美元。

> So, for those of you who have known me for a while, you know that time is very, very important to me. And one of the papers that has had a big impact on my life and my career is this concept of these killer microsconds. Um, in this paper, you know, they talk about we're good at handling the, you know, millisecond problems and the nancond problems, but we're not good at handling the microscond problems. And today the AI workloads, they work in microsconds. So if we can remove those microsconds in terms of latency, we can you know reduce inference cost and training cost and the tokens per watt go up and potentially save the data center providers or the AI providers billions of dollars.

> **小结**　"致命微秒"这个提法的关键不在"微秒很小"，而在"工程投入的分布恰好绕开了它"——毫秒有操作系统管，纳秒有电路设计管，微秒落在两者之间的空白带。

03 高基数把网络拍平



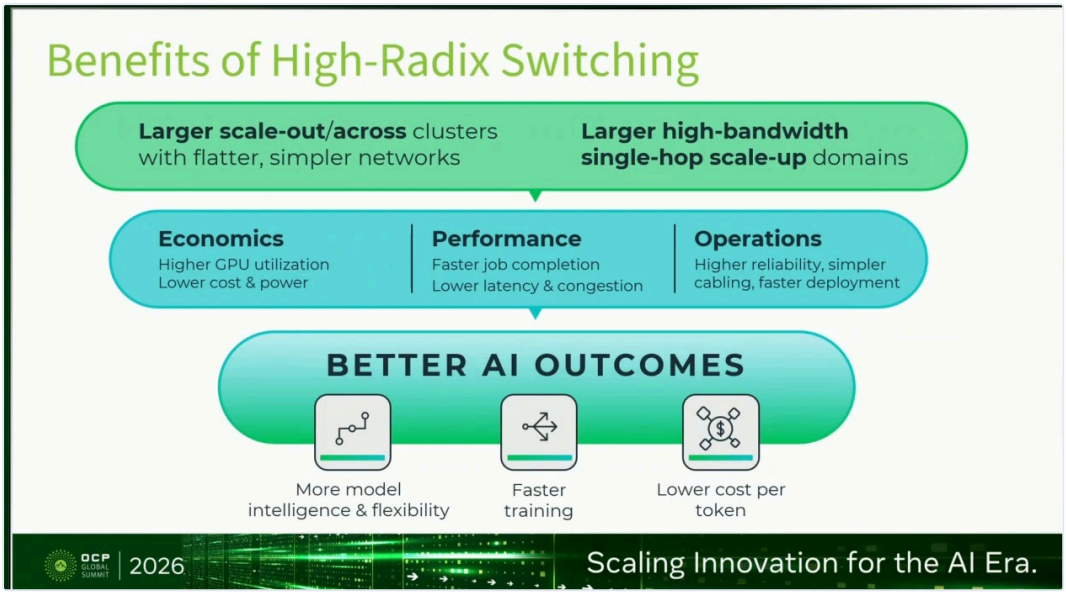
High Radix Switching Flattens Networks and Delivers Massive Latency Benefits：左侧 30 台 102T 交换机、640 个 1.6T 端口的多跳拓扑，右侧假想的单台 1P 交换机单跳，下方列出延迟收益

我在一家叫 Eridu 的公司工作。离开上一家雇主时，我想去一个能攻击这些"致命微秒"的地方。在 Eridu，我们就在攻击这些致命微秒，我很喜欢我的工作。要说明我们如何攻击这些致命微秒，可以看一个假想的 1 Pbps 系统：用 102T（太比特每秒）的交换机，需要大约 30 台这样的交换机才能实现 AI 所需的这个巨大带宽。但如果我们看一台假想的、单一 1 Pbps 的交换机，我们一跳就能做到。而当我们能减少这些跳数，交换延迟就会显著下降，同时还带来其他好处——减少抖动、减少拥塞、减少尾延迟。

> And so I work for a company called Ir. And when I left my previous employer, I wanted to go somewhere where we could attack those killer microsconds. And at IDO we are attacking those killer microsconds. And I love my job. And so to show you how by itself we are attacking those killer microsconds is if you look at a hypothetical one pabit you know system with 102 t or terabits per second switches. It takes about 30 of those switches to implement uh this bandwidth this massive bandwidth that AI needs. But if we look at a hypothetical single one pedabit switch, we can do it in a single hop. And when we can reduce those hops, then the switch latency gets reduced significantly, which also has other benefits in terms of reducing jitter, congestion, and tail

小结　这张图的对比是"30 台设备 + 多层互联"对"1 台设备 + 640 个 1.6T 端口"：总带宽相同，差别全在跳数上——而跳数正是那几微秒的来源。

04　高基数带来的三重收益



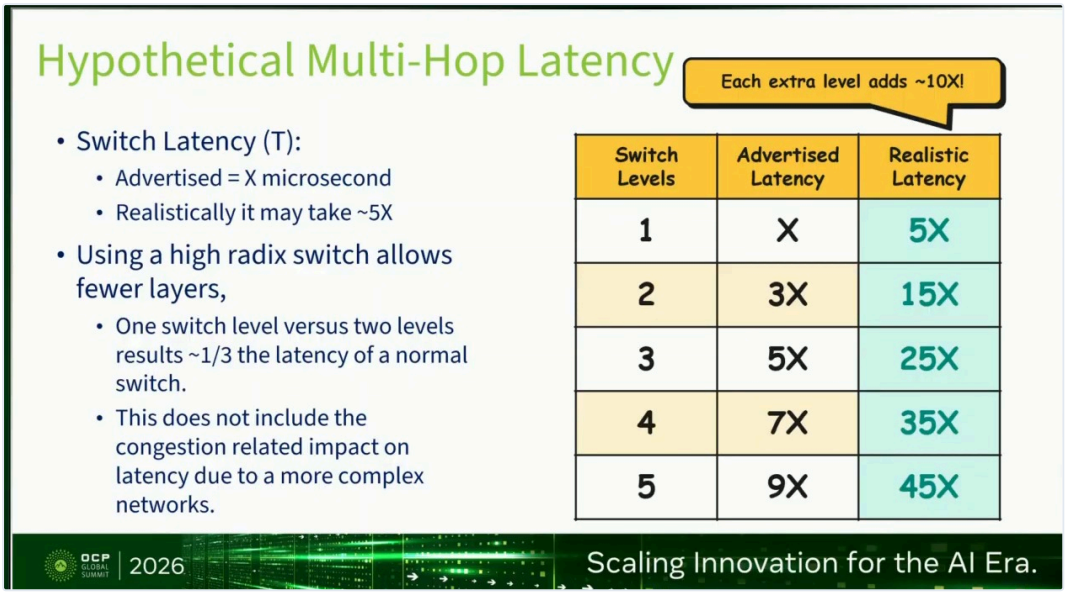
Benefits of High-Radix Switching：更大的 scale-out/scale-across 集群与更平的单一跳数 scale-up 域，进而带来经济性、性能、运维三方面改善，最终是更好的 AI 结果

……尾延迟。所以高基数交换机不仅有利于 scale-up 和 scale-out，也有利于 scale-across：经济性、性能、运维都会更顺，最终带来更好的 AI 结果——更快的训练、更低的每 token 成本。那我们想想这件事。如果我……

> latency. And so a high rate switch can benefit not only scale up and scale out but also scale across the economics, the performance and the operation all will be smoother and it leads to better AI outcomes, the faster training, the lower cost per token. So let's think about this. If I

> **小结**　这页把收益拆成三层：拓扑层（更平）、运营层（经济性/性能/运维）、结果层（模型能力/训练速度/每 token 成本）——注意最上层写的是"更多模型智能"，这是网络演讲里少见的表述。

05　多跳延迟的账：每多一层约 10 倍



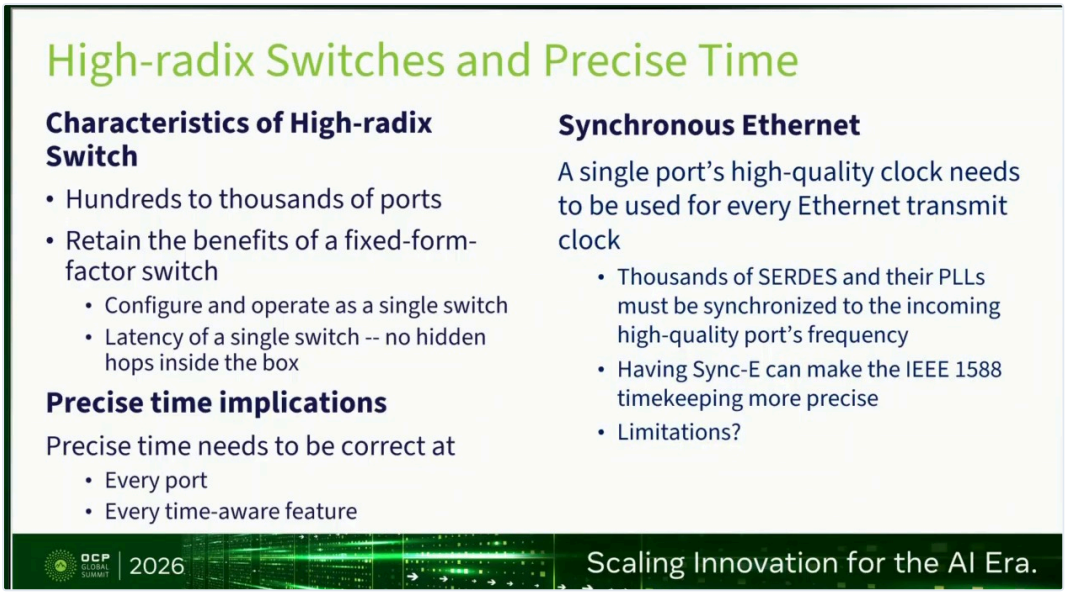
Hypothetical Multi-Hop Latency：交换机延迟 T，标称 X 微秒、实际约 5X；跳数 1–5 层对应实际延迟 5X / 15X / 25X / 35X / 45X，标注 "Each extra level adds ~10X!"

想想今天数据中心里有什么：我有 top-of-rack 交换机、leaf、spine。如果我能压缩掉其中一层，我就能降低延迟。所以如果我能从两层的交换变成一层的交换，我就能把延迟降低三分之一。所以如果每一层的延迟是几微秒，我每层都要付这几微秒，而如果我能压缩成单层，我节省的就是其中三分之一。

> think about what's in the data center today, right? I have a to a top of rack switch a leaf and a spine. And if I can compress one of those layers then I can reduce the latency. So if I can go from say two layers of switching to one layer of switching I can reduce my latency by a third. And so if you know my my latency is say a couple of microsconds right for each layer I'm going to hit that couple of microsconds but my savings will be you know onethird of that if I can compress it into a single layer

> **小结**　幻灯片右上的标注是全场最尖锐的一句：标称延迟和实际延迟差 5 倍，而"每多一层还要再乘约 10 倍"——这意味着多跳网络的真实成本远比标称值难估算。

06 高基数交换机与精确时间



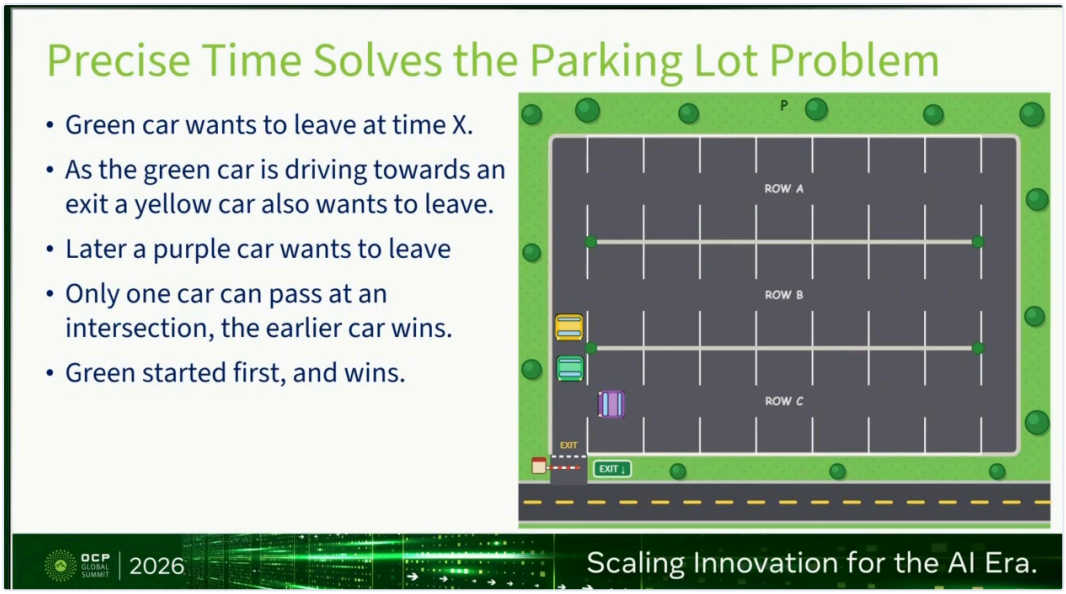
High-radix Switches and Precise Time：左侧高基数交换机特征（数百到数千端口、像单台交换机一样配置运行、无隐藏跳数）与精确时间含义（每个端口、每个时间感知特性），右侧同步以太网（SyncE）的要求与局限

那我们现在谈精确时间。这种高基数交换机的特征是什么？它会有几百到几千个端口，不是几十个端口或一百出头，而是几百到几千个，甚至更多。而且它具备固定形态交换机的全部好处：它像单台交换机一样被配置和运行，延迟就是单台交换机的延迟，机箱内部没有隐藏的跳数。而这对精确时间的影响是巨大的。今天大多数精确时间方案面向的是几十个端口，也许几百个端口，但现在我们要面对几千个端口，并且要让所有不同端口之间的抖动尽可能低。不仅如此，每一个时间感知特性都需要那个精确时间。在此之上还有同步以太网：我们要从一个端口引入信号，把它净化，然后送到几千个端口去。

> and so let's talk about now precise time. So what are the characteristics of this high radic switch? Well, it's going to have hundreds to thousands of ports, right? It's not going to be tens of ports or low hundreds. It's going to be hundreds to thousands of ports, maybe even more, right? And it's going to have the same benefits as a fixed form factor switch, right? It it configured and operated like a single switch. The latency is of a single switch. There's no hidden hops inside the box. And the implications of this on precise time is huge. Most of the the precise time solutions that we have today are targeting, you know, tens of ports or maybe hundreds of ports, but now we're looking at thousands of ports and keeping the jitter all, you know, as low as possible between all the different ports. And not only that, every time aware feature needs to be have that precise time. And so on top of that is synchronous Ethernet, right? We're going to pull in a signal from one port, we're going to clean it up, and then we're going to send it to thousands of ports.

小结　这页提出了一个容易被忽略的量化要求：精确时间不是"整机对准一次"，而是"每个端口、每个时间感知特性都对准"，端口数从百级跳到千级，意味着同步树的分支数也跳了一个量级。

07 精确时间解决停车场问题



Precise Time Solves the Parking Lot Problem：绿车在时刻 X 想离开，黄车也要离开，路口只能过一辆，先到的赢；右侧为 A/B/C 三排停车位示意

所以当我思考高基数交换机的问题时，它是很有物理挑战性的，是个非常大的问题。而这个"停车场问题"是我们已经能用精确时间解决的一个问题。这里的想法是：你可以把所有输入端口的精确到达时间作为调度决策的一部分。你由此得到的公平机制或度量是——你要把到达时间和驻留时间考虑进去，并且你要降低尾延迟，而所有人都希望尾延迟尽可能低。我们走一个例子。假设你在停车场，我们把到达时间当作一辆车想离开的时间。在这个例子里，绿车想在时刻 X 离开。当它正往外开的时候，突然黄车也想离开。现在我们处在绿车和黄车的路口。如果黄车赢了，它就增加了绿车的尾延迟。所以如果我们知道绿车是什么时候开始走的，为了公平，我们可以基于到达时间给绿车优先权。果然，我们让绿车先走。现在我们到了另一个路口，有绿车和紫车。紫车是最后一辆到的，所以如果它先走，它就会超过绿车，让绿车慢下来，增加它的尾延迟。黄车同理。所以我们会让绿车先通过，然后黄车通过，然后紫车通过。这样它们都获得我们能给的最低尾延迟。你看着这个会说，Dan，这很简单，这没什么，这是小事。但当你从高基数的角度看，这些停车场一下就变大了：现在不是几十个车位……

> And so when I think of the the the problem of a highic switch, it's physically demanding, right? It's a very large problem and you know this parking lot problem is one that we've been able to solve with precise time and the idea here is that you can use the precise arrival time at all the inputs as part of your scheduling decision right and the fairness mechanisms or metrics that you'll get out of this is you know you want to have you know arrival time and residency time considered and you want to reduce that tail latency which everybody wants to be as low as possible. And so let's go through an example. So let's say you you're in the parking lot and we are going to use the arrival time as a time where a car wants to leave. In this case the car wants the green car wants to leave at time X, right? And as it's leaving all of a sudden the yellow car wants to leave. So now we're at an intersection between the green car and the yellow car. If the yellow car wins, it adds to the tail latency of the green car, right? So if we know when the green car started to leave, to be fair, we can give, you know, preference to that green card based on its arrival time. And sure enough, we'll let the green car go ahead. Now we hit another intersection and we have the green car and the purple car. The purple car was the last car. So if it went first, it would get ahead of the green car, slowing it down, adding to its tail latency. And same for the yellow car. So we're going to let the green car go through, then the yellow car go through, and then the purple car go through. And this way they all have the lowest possible tail latency that that you know we can give them. And so

> you look at this and oh Dan this is easy you know this is nothing right this is you know small but when you look at things in terms of high radex all of a sudden those parking lots get a little bit bigger so now instead of you know tens

小结　"先到先走"听起来是常识，但要执行它，交换机必须知道每个包**何时到达**——这个时间戳的精度直接决定了调度是否真的公平，这也是精确时间与交换芯片的接合点。

08　当停车场有 5000 个车位



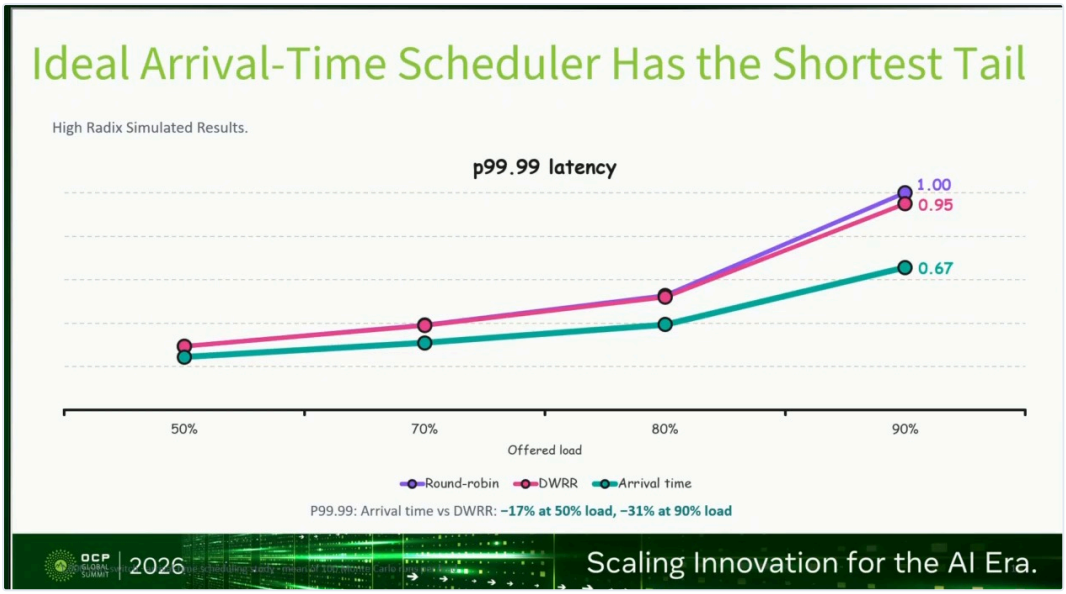
~5000 Spaces：一张密密麻麻的停车场示意图，数千个车位与出口

……个车位，现在我有几千个车位，或者可能有 5000 个车位。所以精确时间提供了一种方式，让所有这些不同的"车"、也就是进入网络设备的各个输入，有一个共同的基础可以依据。而且并不是所有这些车都去一个出口——这个系统里其实有 5000 个出口。所以我们必须调度 5000 个不同的入口和 5000 个不同的出口。于是我做了一些仿真，看……

> of spaces now I have thousands of spaces or potentially 5,000 spaces and so precise time gives a way to give all of these different cars or inputs into a networking device, you know, a a a base that they can work off of, right? And it's not that we're all these cars are going to one exit. There's actually 5,000 exits in this system. And so we have to schedule, you know, 5,000 different entrances and 5,000 different exits. And so I ran some simulations on what

小结　这张图的作用是把直觉撞碎：小停车场里"先到先走"靠人眼就能判断，5000 个车位 × 5000 个出口时，公平性只能靠硬件时间戳和调度算法来保证。

09　理想到达时间调度器的尾延迟最短



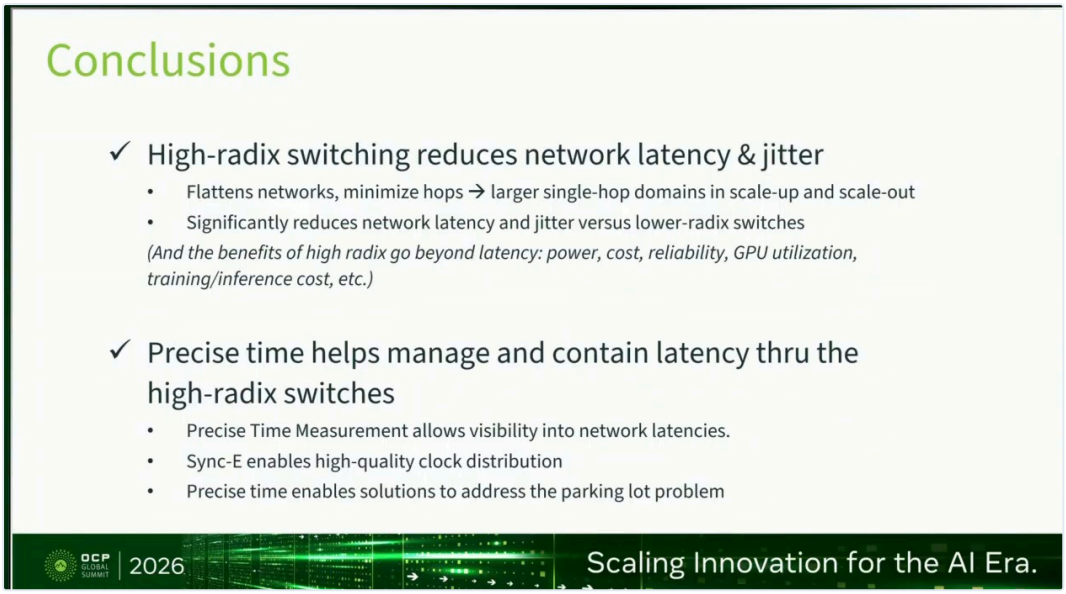
Ideal Arrival-Time Scheduler Has the Shortest Tail：p99.99 延迟随负载变化，Round-robin 1.00、DWRR 0.95、Arrival time 0.67；脚注 P99.99 相比 DWRR 在 50% 负载降 17%、90% 负载降 31%

……一个 1000 端口系统会是什么样。我想比较轮询（round-robin）、亏空加权轮询（DWRR）和一个到达时间调度器，也就是理想的到达时间调度器。轮询就像你在一个四向停车标志前，在四个方向之间轮转，谁先到谁先走。亏空加权轮询基本上是在所有方向之间尽量均等，果然 DWRR 的性能会好一点，它在今天的产业里用得相当普遍。但如果我们有一个理想的到达时间调度器，我们一下就把延迟降到大约 67%——这是归一化延迟，而且这是你的 P99.99 延迟。所以如果原来是 3 微秒延迟，我就这样干掉了一微秒。

> this would look like for a thousandport system. And I wanted to compare a roundroin, a deficit weighted roundroin and an arrival time, an ideal arrival time scheduler. And so round robin is like you're at a four-way stop sign and you rotate between the four different uh you know people at the stop sign. Whoever got there first gets to go. deficit weighted round robin is you basically try to equal it between all the different um all the different uh directions and sure enough you get a little better performance with DWRR and it's used quite commonly in industry today but if we had an ideal arrival time scheduler all of a sudden we've reduced it to about 67 you know percent of this nomal nominalized uh latency and this is your P99.99 latency. So if it took you know three microsconds of latency you know I've just killed a microscond

> **小结**　这是全场唯一的实测（仿真）数据：DWRR 相对轮询只改善 5%，而到达时间调度相对 DWRR 在 90% 负载下改善 31%——说明调度策略的收益远大于同策略内的微调。

10　结论：两条腿



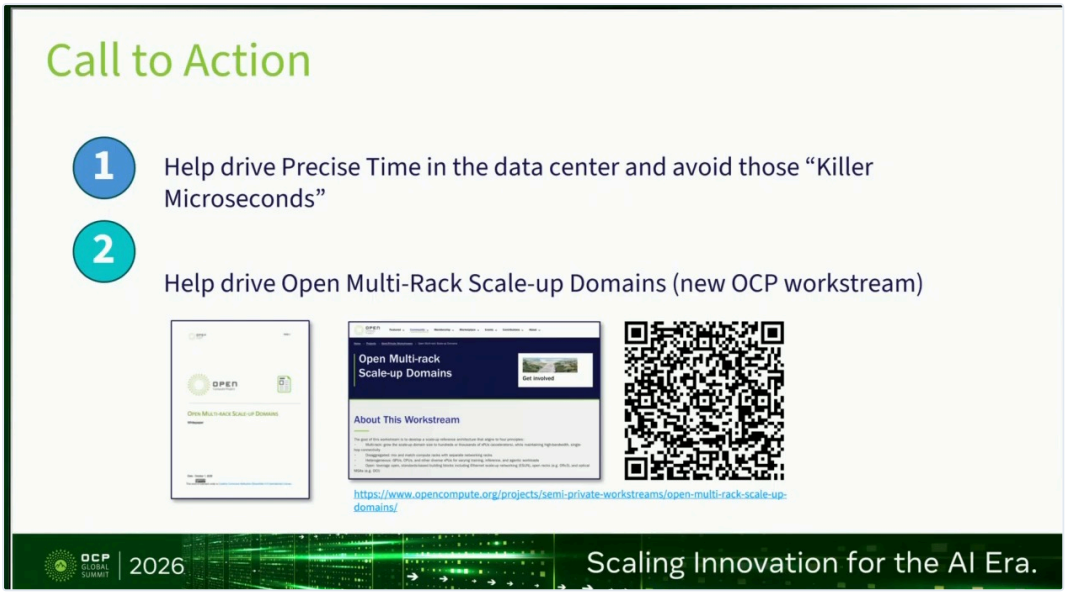
Conclusions：高基数交换降低网络延迟与抖动；精确时间帮助管理并约束高基数交换机内的延迟（精确时间测量带来可见性、SyncE 分发高质量时钟、精确时间使停车场问题可解）

所以结论是：高基数交换机降低网络延迟与抖动，它把网络拍平、最小化跳数，在 scale-up 和 scale-out 中形成更大的单跳域，相对低基数交换机显著降低网络延迟与抖动。而且收益不止于延迟——还有功耗、成本、可靠性、利用率、训练和推理成本。而精确时间帮助管理并约束穿过高基数交换机的延迟。精确时间测量——我今天看这个的时候注意到，不是 PTM，而只是测量精确时间——让你在许多不同点上获得对这些延迟的可见性。SyncE 让你分发一个非常高质量的时钟。而精确时间让方案能够解决像今天这个停车场问题的问题。所以你今天回家路上经过那些路口时会想：嘿，我先到的，本来应该我先走。总之……

> and so you know for conclusions the high rate switch you know reduces network latency and jitter it flattens minimize hops you know it's a larger single hop domain and scale up and scale out it significantly reduces the network latency and jitter versus low radex switches. And the benefits go beyond, you know, the the latency, right? There's power, there's cost, there's reliability, utilization, training, and inference costs. And precise time helps manage and contain latency through the high switch, right? So precise time measurement and I saw this today as I was looking at this not the PTM but just measuring precise time you know allows visibilities into these latencies at many different points right sinky allows you to distribute a very high quality clock and precise time enables the solution to address problems like today's parking lot problem and as you go home today when you're at that different intersections you'll see you know hey I I started first. I should have been able to go first. But anyway,

小结　结论页把两件事明确分工：高基数负责"减少延迟的来源"（跳数），精确时间负责"让剩下的延迟可管理"（调度与可见性）——缺一条腿都不成立。

11　行动号召与问答：单点故障怎么办



Call to Action：一是推动数据中心内的精确时间、避开"致命微秒"，二是推动开放的多机架 scale-up 域（OCP 新工作流），附白皮书封面、工作流页面与二维码

行动号召。我觉得我这辈子的使命就是试着干掉那些"致命微秒"。客户花了很多时间写论文、发表论文，讲干掉致命微秒的好处。所以在 Eridu，我们正通过把 AI 网络拍平来干掉这些致命微秒。在此之上，我们认为 scale-up 网络会增长：如果我有一台几千个端口的交换机，那我可能会把它连到几千个 GPU 上，那不可能装进一个机架。所以今天早些时候有人谈到"机架级时间"。但我认为我们真的需要从 **scale-up 域时间**的角度来思考这件事。而这可能只是这个多机架 scale-up 域的一个方面——已经有一些公司（包括我们）开始组织这件事。如果你想加入这个工作流，可以拍一下二维码，了解我们在做什么。那里还有一份白皮书。谢谢。

主持人：谢谢 Dan。观众有问题吗？好。

提问者：麦克风开了吗？喂？现在能用了。好，谢谢。我的问题是：如果你把这么多端口塞进一个机箱，什么能阻止它变成单点故障？

Dan Biederman：你完全说对了。所以当我们走向多平面（multi-plane）机箱时，我们会有多个平面。你的 GPU 或网卡会连接到多个平面。所以如果我有 1000 个 GPU，我可能有比如 5 台或 10 台交换机，每台建立不同的连接，然后我把流量喷洒到这些不同的端口上。现在他们在谈 MRC，它跑在 SRv6 之上，基本上就是让网卡把流量喷洒到多个平面上。所以如果突然有一台交换机挂了，爆炸半径被最小化，影响被最小化。这次峰会上早些时候也有一些演讲谈到 MRC：你可以看到性能下降，然后在 MRC 里出问题时立即响应。谢谢。

主持人：还有别的问题吗？好。谢谢 Dan。谢谢。

> call to action. So, you know, I feel my mission in life is to try and kill those killer microsconds. A customer spend a lot of time to write a paper and publish a paper on, you know, the advantages of killing those killer microsconds. And so here at Erdo we are trying to kill those killer microsconds by flattening the the AI networks. Um on top of that we look at the scaleup networks are going to grow right if I have a switch with thousands of ports then potentially I'm going to connect that to thousands of GPUs that's not going to fit in a single rack. And so earlier today, you know, there was talk about Iraq based time, right? But I think we really need to think of this in terms of the scaleup domain time. And you know,

> that might be one aspect of, you know, this multirack scaleup domain that a few companies including are starting to you know to to organize. And if you'd like to join that workstream, you know, you can take a picture of the the uh QR code and learn more about what we're doing. There's also a white paper there. Thank you. Thanks, Dan. Do we have any questions from the audience? Yes. Is the mic microphone on? Hello. Hello. It's working now. Good. Thank you. Anchor. Um, okay. So my question is if you're piling so many ports into a single box, what stops that being a single point failure? You're absolutely right. Right. So as we move towards multiplaner boxes, right? We will have multiple planes. So your your GPUs or your nicks will be connected to multiple planes, right? So, if I have a thousand GPUs, I may have say five or 10 switches, each one of those with a different connection, and I'm going to spray my traffic across those different ports. Right? Right now, they're talking about MRC, uh, which runs off of SRV6, where they'll basically have the nick spray the traffic across multiple planes. So, if all of a sudden your your switch dies, the blast radius is minimized, right? The effect is minimized. And there were some presentations um earlier in this uh this u summit where they talked about how MRC6 you could see a degradation and then an immediate response as bad things happened in the MRC. And so thank you. Any other questions? Okay. Uh thanks Dan. Thank you.

> **小结**　问答暴露了这场论证的薄弱环节：整场讲"更大的单跳域"，但单机箱端口数越多，单点故障的爆炸半径越大；他的答案是靠多平面加流量喷洒，也就是用外部冗余来补内部集中的风险。

附录

附录 A　数据核对表

项目	数值 / 结论	来源	备注
演讲时长	13 分 20 秒（800.5 s）	视频元数据	—
对比拓扑	30 台 102T 交换机、640 个 1.6T 端口 vs 1 台假想 1P 交换机、640 个 1.6T 端口	幻灯片 03	640 × 1.6T ≈ 1.02P
多跳延迟（1–5 层）	标称 X / 3X / 5X / 7X / 9X；实际 5X / 15X / 25X / 35X / 45X	幻灯片 05	幻灯片标注 "Each extra level adds ~10X!"
单层压缩收益	两层压成一层，延迟降约三分之一	口播	—
仿真规模	1000 端口系统	口播	—
p99.99 延迟（90% 负载，归一化）	Round-robin 1.00 / DWRR 0.95 / Arrival time 0.67	幻灯片 09	与口播"降到约 67%"一致
到达时间调度 vs DWRR	50% 负载 −17%，90% 负载 −31%	幻灯片 09 脚注	—

项目	数值 / 结论	来源	备注
停车场规模	~5000 个车位、5000 个出口	幻灯片 08	—
引用论文	Attack of the Killer Microseconds，DOI 10.1145/3015146	幻灯片 02	Patterson & Hanrahan，CACM
新工作流	Open Multi-Rack Scale-up Domains（OCP）	幻灯片 11	附二维码与白皮书

> **一处需要说明的算术**　幻灯片 03 左侧写"30 × 102T switches / 640 × 1.6T ports"。若按每台 102T 计，30 台合计约 3.07P，与口播的"1 Pbps 系统"不符；而 640 × 1.6T = 1024T ≈ 1P，与"1 Pbps"吻合。因此更合理的读法是：**640 个 1.6T 端口是整个 fabric 的总端口数，1P 是总带宽**，30 台是把这个带宽拆成多跳的方式。正文按幻灯片原文陈述，此处标注差异。

附录 B　时间轴

时间	内容
00:00	议程与免责声明
00:33	致命微秒：AI 负载工作在微秒尺度
01:23	30 台 102T 交换机 vs 单台 1P 交换机
02:23	高基数对 scale-up / scale-out / scale-across 的收益
02:46	从两层交换压到一层，延迟降三分之一
03:34	高基数交换机的特征与对精确时间的含义
04:55	停车场问题：用到达时间做调度
07:07	5000 个车位与 5000 个出口
07:47	仿真：轮询 / DWRR / 到达时间调度
08:57	结论：高基数 + 精确时间
10:14	行动号召、多机架 scale-up 工作流、问答
13:08	结束

附录 C　术语表

缩写	全称	说明
Radix	—	交换机端口数量级；高基数指数百至数千端口
Scale-up	—	纵向扩展，单域内更多加速器互联
Scale-out	—	横向扩展，更多节点组成集群
Scale-across	—	跨域扩展，多个集群互联
DWRR	Deficit Weighted Round-Robin	亏空加权轮询调度
p99.99	—	99.99 分位延迟，尾延迟指标
SyncE	Synchronous Ethernet	同步以太网，从线路恢复频率
PTM	Precision Time Measurement	PCIe 精确定时机制
MRC	—	多平面流量喷洒机制，运行于 SRv6 之上
SRv6	Segment Routing over IPv6	IPv6 分段路由
NIC	Network Interface Card	网卡